Fine-tuning a GPT-2 recommender with RL against a learned reward model

First-iteration results on Yambda-50M and KION

Daniil Antonov 11 October 2026

Working report. It covers the three runs completed so far.

Summary. A GPT-2 next-item recommender was fine-tuned with policy gradients to raise the played ratio predicted by a second GPT-2, the reward model. The pipeline ran end to end three times: twice on Yambda-50M with REINFORCE, and once on KION with PPO, where the baselines, the non-RL alternatives and the confidence intervals also ran for the first time.

1. **The base models are sound on both datasets.** On Yambda the recommender reaches NDCG@10 0.114 against 0.019 for popularity, and the reward model predicts held-out feedback with AUC 0.87. On KION the recommender reaches 0.122 against 0.090 for popularity: level with GRU4Rec, above SASRec (0.108), below BERT4Rec (0.129). The KION reward model reaches AUC 0.76.
2. **RL raises the predicted reward as far as the anchor allows and pays with relevance.** With no anchor to the base recommender every user receives the same ten items on both datasets.
3. **With an anchor that holds, the result is the intended one.** On KION at KL weight 0.3 the predicted watched share rises by 23%, and a reward model trained on other users confirms it. NDCG@10 on items the user went on to watch is unchanged (+1%), recall of items the user abandoned falls by 20%, overall NDCG@10 falls by 11%. Bootstrap intervals over users separate every one of these changes from zero except the +1%.
4. **How abruptly the anchor fails differs between the settings.** On Yambda with REINFORCE the result flips between KL weights 0.1 and 0.3 from collapse (NDCG@10 −89%) to a mild shift (−1.7%). On KION with PPO the trade-off is smooth. Dataset and optimiser changed together, so the cause is not separated yet.
5. **On KION RL is not needed for a moderate gain.** Re-ranking the recommender's top 100 candidates with the same reward model is at least as relevant as RL at equal predicted reward up to about +30%, and one epoch of supervised training on liked items lands on the main RL run. RL only reaches further, into the region where relevance on liked items is already down by a fifth.
6. **Each reward is gamed through the item attribute it correlates with.** On Yambda the policy moves to short obscure tracks. On KION it moves from series to films: the predicted watched share rises by 23% while the predicted watch time falls by 11%. The predicted reward of a generated list also depends on what the reward model is told about the items generated before, by up to 14 points on Yambda and 25 on KION.

Not run yet: PPO, the baselines and the non-RL alternatives on Yambda, and more than one seed.

1 Task and setup

The task has three steps: train a GPT-2-architecture recommender for next-item prediction with item identifiers as tokens; build a reward model that predicts the user's response to a recommended item given their history; fine-tune the recommender with RL towards sequences with a higher expected reward, as judged by the reward model. The work follows two papers. Petrov and Macdonald [1] fine-tune GPTRec with PPO towards a reward that combines NDCG with diversity or popularity terms.

Matveev et al. [2] train a retriever on Yambda-5B with off-policy REINFORCE on logged sessions and use a learned user feedback model for evaluation.

1.1 Data

The first dataset is Yambda-50M [3], the listening log of Yandex Music: 46.5 million listens of 10,000 users over about 300 days. Each event has the track, a timestamp, the share of the track that was played, the track length, and a flag telling whether the user chose the track or a recommender served it. Preprocessing does the following.

- **Sessions.** An event "continues the session" if the same user has another event within 30 minutes.
- **Split.** One global time axis: the last 7 days are the test period, the 7 days before are validation, the rest is training. Everything derived from data (catalogue, popularity, item statistics) uses the training period only.
- **Catalogue.** The most popular tracks of the training period become tokens, numbered by popularity; events on other tracks are dropped. Users need at least 50 training events.

With a 50,000-track catalogue this keeps 37.2 million events (80.1%) of 8,711 users: 35.0 million for training, 1.13 million for validation and 1.08 million for testing. The mean played ratio is 0.65; 64% of events are played at least half; 95% of events continue the session; 51% of events are organic. The played ratio is bimodal (Figure 1): a track is either skipped early or played to the end, so the reward model is in effect a skip classifier. Short tracks are finished somewhat more often than long ones, 0.70 against 0.60 between the extreme duration deciles; popularity makes no visible difference.

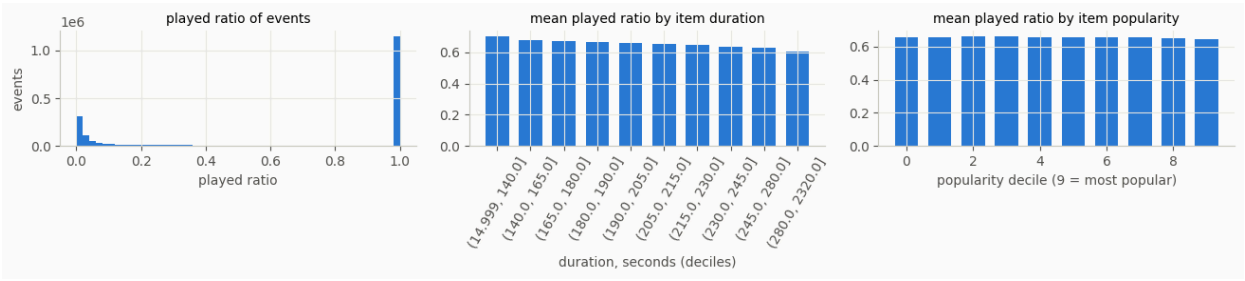


Figure 1: The reward signal in the processed Yambda log (50,000-track catalogue). Left: distribution of the played ratio. Middle and right: mean played ratio of tracks by duration and by popularity decile.

The second dataset is KION [4], the log of the MTS online cinema: 5.5 million rows of 962,000 users over about five months. It differs from Yambda in ways that matter for every step. There is one row per user and title, holding the date of the last viewing, the total time watched and the share of the title that was watched; that share is the played ratio here. Timestamps are dates. Nothing says whether a title was recommended. Histories are short. The same preprocessing is applied with thresholds that fit: a title needs 10 training events and a user 5, and an event "continues the session" if the user watches another title within 7 days. Table 1 puts the two processed logs side by side.

Table 1: The two processed logs.

	Yambda-50M (run A)	**KION**
Events kept	37.2 M (80.1%)	3.88 M (70.9%)
Users	8,711	256,407
Items in the catalogue	50,000	7,992
Events per user, mean	4,270	15
Training / validation / test events	35.0 M / 1.13 M / 1.08 M	3.51 M / 0.19 M / 0.18 M
Mean played ratio	0.65	0.50
Events played at least half	64%	48%
Events that continue the session	95% (30 minutes)	80% (7 days)
Test users with a request	7,593	59,997

The watched share on KION is bimodal too (Figure 2), but it depends on the length of the title far more than the played ratio of a track does. Titles of film length, about 80 to 140 minutes, are watched to 0.5 on average. The longest tenth of the catalogue, more than 8.5 hours of content and therefore

series, is watched to 0.22, because the share is taken of the whole series. A reward built on this share prefers films before any model is involved.

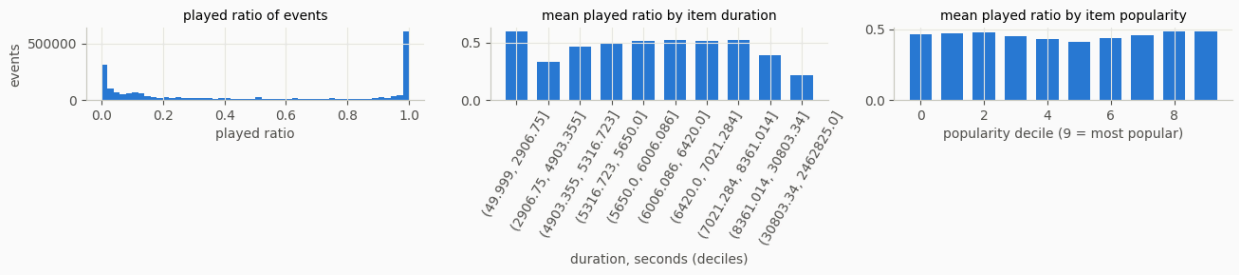


Figure 2: The reward signal in the processed KION log. Left: distribution of the watched share. Middle and right: mean watched share of titles by duration and by popularity decile.

1.2 Models

Base recommender. A GPT-2 decoder (Hugging Face implementation, 4 layers, width 256, 4 heads, dropout 0.1) over track tokens, with tied input and output embeddings. It is trained on windows of 128 consecutive events of one user with full-softmax cross-entropy on the next track, AdamW at learning rate 10^{-3} with cosine decay, and early stopping on the validation loss. It produces recommendations in two ways. *Ranking* (top-k) scores all tracks once and takes the best. *Generation* (next-k) produces ten tracks one at a time, feeding each choice back as input and never repeating a track within the list. RL changes the second. On KION the windows are 48 events long, dropout is 0.2, and titles in the user's history are never recommended, because each user and title occur once.

Reward model. A second GPT-2 decoder of the same size. The token at position t carries track t together with the played ratio of track $t-1$ (in ten buckets), so the output at position t has seen the candidate track and the full history but not the response it predicts. Two heads predict the played ratio (cross-entropy with the ratio as a soft label) and whether the session continues. Two copies are trained on disjoint halves of the users. The *train* reward model produces the rewards during RL. The *held-out* reward model never influences the policy and only judges its lists: a gain it does not confirm is exploitation of the errors of the first.

RL fine-tuning with REINFORCE (runs A and B). Each step samples a batch of recommendation moments from training histories (a user and a cut point), and for each one several lists of ten tracks from the current policy. The train reward model predicts the played ratio of every track of every list, position by position; for tracks that were generated it feeds its own expected ratio back as their feedback. The reward of a track is its predicted played ratio, and its return is the sum of the rewards from its position to the end of the list. The baseline of a list is the mean return of the other lists sampled for the same history, which leaves the gradient unbiased. Advantages are scaled to unit variance. The loss is

$$\mathcal{L}(\theta) = -\mathbb{E}[A \cdot \log \pi_\theta(a \mid s)] + \beta \cdot \mathrm{KL}(\pi_\theta \,\|\, \pi_0),$$

where π_0 is the frozen base recommender and the KL term is computed exactly over the catalogue at every position. This is REINFORCE with a baseline and a KL anchor. The reward contains no relevance term, so the anchor weight β is the only thing that asks the policy to stay relevant.

RL fine-tuning with PPO (run C). The baseline is replaced by a value model: a copy of the recommender's transformer with a scalar head, trained to predict the return from every position of a list. Advantages come from generalised advantage estimation (discount 0.99, $\lambda = 0.9$) and are scaled to unit variance. Each iteration samples one list for each of 512 histories and takes four gradient steps on it with the clipped objective of PPO (clip 0.2); the same KL anchor is added. The policy learning rate is 10^{-5} and the value model's is 10^{-4}; for the first 50 iterations only the value model is trained. Dropout is off during fine-tuning.

Baselines (run C). Most popular items, a first-order Markov chain, GRU4Rec with a full softmax, SASRec with its original loss (binary cross-entropy against one sampled negative per position) and

BERT4Rec trained on the cloze task. The neural ones take the recommender's width, depth, dropout and window length, train on the same windows and stop on their own validation loss. None is tuned.

Alternatives to RL (run C). Two cheaper ways of raising the reward. *Liked-only fine-tuning* continues supervised training of the recommender for one epoch, but only towards items the user played at least half of; it uses no reward model. *Re-ranking* takes the recommender's top 100 candidates and orders them by $w \cdot$ (the recommender's log-probability) + (the reward predicted by the train reward model), for several weights w; it needs no training. Each candidate's reward is predicted as the next item after the real history.

1.3 Evaluation

Every test user who has history issues one request at the start of the test week: the last 118 events (38 on KION) are the history, the next 20 events at most are the truth. There are 7,593 such users on Yambda with the 50,000-track catalogue and 59,997 on KION. Three groups of measurements are taken.

- **Relevance.** Recall, NDCG, MAP and hit rate at 1, 5 and 10, against all 20 future tracks, against those played at least half (*liked*), and against the rest (*skipped*). The split needs no reward model: a policy that truly learned to avoid skips should lose relevance on skipped tracks and keep it on liked ones.
- **Predicted reward.** Both reward models score the generated lists.
- **Collapse.** Catalogue coverage, the share of users who receive the most frequent track, the expected overlap between the lists of two users, the share of recommended tracks already in the history, and the popularity and duration of recommended tracks.

In run C the change from the base recommender to the fine-tuned one comes with a 95% interval from a paired bootstrap over users. It measures the noise of the user sample, not the noise between training runs.

1.4 The three runs

Table 2 lists what differed. Run B was sized for a much weaker GPU; apart from the catalogue it is run A with less training. Run C changes the dataset and the optimiser at once, and is the first run of the baselines, the alternatives to RL and the intervals.

Table 2: The three completed runs. All use a horizon of 10 items. Yambda numbers in this report are from run A unless stated otherwise.

	Run A	**Run B**	**Run C**
Dataset	Yambda-50M	Yambda-50M	KION
GPU	RTX PRO 6000, 96 GB	Tesla T4, 16 GB	RTX PRO 6000, 96 GB
Catalogue	50,000 tracks	30,000 tracks	7,992 titles
Events kept; users	37.2 M; 8,711	33.3 M; 8,669	3.88 M; 256,407
Recommender training	15 epochs, batch 256, windows of 128	6 epochs, batch 64, windows of 128	11 epochs (early stop), batch 512, windows of 48
RL algorithm	REINFORCE	REINFORCE	PPO
RL main run	4,000 steps of 1,024 lists	1,500 steps of 256 lists	2,000 iterations of 512 lists
Lists per history	4	4	1
RL sweep runs	2,000 steps each	500 steps each	1,000 iterations each
KL weight, main run	0.1	0.1	0.3
KL weights in the sweep	0, 0.01, 0.03, 0.1, 0.3, 1, 3	0, 0.03, 0.3, 3	0, 0.1, 0.2, 0.3, 0.5, 1
RL learning rate	$3 \cdot 10^{-5}$	$3 \cdot 10^{-5}$	10^{-5}
Baselines, alternatives, intervals	no	no	yes
Wall-clock time	9.5 hours	5.0 hours	1.3 hours

2 Yambda-50M with REINFORCE (runs A and B)

2.1 The base recommender

Table 3 shows the recommender after supervised training. It is far above the popularity floor on every measure. Unlike in Petrov and Macdonald [1], generating the list item by item does not hurt: NDCG@10 is slightly higher than with ranking. Their truth is the single next item, so every later position of a generated list is conditioned on a wrong prefix. Here the truth is the next 20 events, and a model trained to continue sequences is asked to do exactly that. Half of what the recommender proposes is already in the user's history, as expected for music.

Table 3: Base recommender on 7,593 test users (run A).

	Most popular	GPT, ranking	GPT, generation
NDCG@1	0.0246	0.1655	0.1655
NDCG@5	0.0182	0.1272	0.1329
NDCG@10	0.0186	0.1144	0.1175
Recall@10	0.0114	0.0743	0.0752
MAP@10	0.0079	0.0627	0.0757
Hit rate@10	0.1118	0.4465	0.3677
NDCG@10, liked tracks	0.0146	0.0995	0.1010
Recall@10, skipped tracks	0.0108	0.0743	0.0737
Catalogue coverage	0.0002	0.4663	0.5234
Share already in history	0.083	0.421	0.505

2.2 The reward model

Both reward models predict held-out feedback well (Table 4) and are close to perfectly calibrated (Figure 3). Two details matter for what follows. First, the user's recent behaviour alone, the mean played ratio of their preceding events, already reaches AUC 0.80, while the track's average played ratio reaches only 0.61. Most of the model's skill is knowing whether this user is currently skipping; what it knows about individual tracks is the smaller part, and that part is what RL optimises. Second, the validation loss was lowest after the first epoch and rose afterwards, so the models used are one-epoch models.

Table 4: Prediction of the played ratio on all test-period events, each predicted from the events before it (run A). AUC is for "played at least half".

Predictor of the played ratio	MAE	RMSE	Cross-entropy	AUC
Train reward model	0.236	0.327	0.433	0.867
Held-out reward model	0.243	0.328	0.434	0.865
Mean of the user's preceding events	0.289	0.370	0.510	0.798
Mean played ratio of the track	0.393	0.429	0.623	0.607
Global mean	0.409	0.434	0.635	0.500

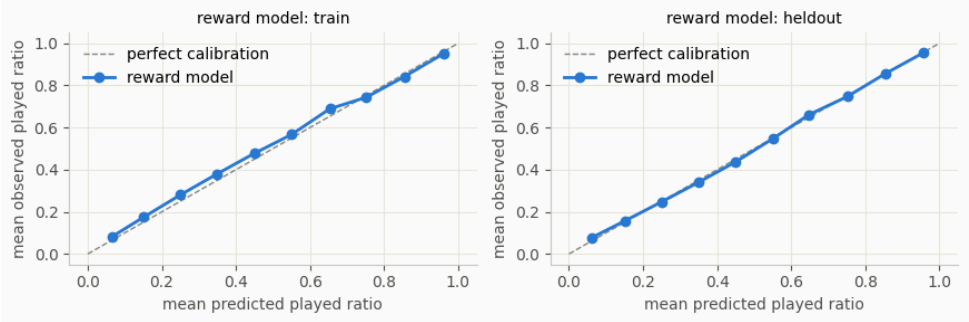


Figure 3: Calibration of the two reward models on test-period events.

2.3 RL with a weak anchor collapses

The main run used a KL weight of 0.1. Table 5 compares the fine-tuned policy with the base recommender.

Table 5: Main RL run of run A (KL weight 0.1, 4,000 steps) against the base recommender.

Generated lists, test users	Base	After RL	Change
NDCG@10, all future tracks	0.1175	0.0125	−89%
NDCG@10, liked tracks	0.1010	0.0123	−88%
Recall@10, skipped tracks	0.0737	0.0059	−92%
Predicted played ratio, train reward model	0.580	0.811	+40%
Predicted played ratio, held-out reward model	0.613	0.713	+16%
Predicted seconds played, held-out reward model	123.1	93.2	−24%
Catalogue coverage	0.523	0.050	
Share of users receiving the top track	0.023	0.821	
Expected overlap of two users' lists	0.001	0.466	
Share already in history	0.505	0.037	
Mean popularity of recommended tracks (plays)	2,943	757	
Mean duration of recommended tracks (s)	203	130	
Mean played ratio of recommended tracks in the log	0.654	0.906	

The policy did maximise the reward it was given, and it stopped recommending. Four observations explain how.

What it moved to. The fifteen tracks that gained the most exposure have 136 to 2,294 plays in the training period (the most popular track has 39,487), last 90 to 165 seconds, and were played 85% to 97% on average by the listeners they had, against 65% overall. The most exposed one is in the lists of 6,235 of the 7,593 users. These are niche tracks with a self-selected audience: the people who found them liked them, and the reward model generalises that to everyone. The policy gives up the tracks the user already knows (the share from history drops from 51% to 4%).

The gain is partly exploitation. The train reward model credits the new lists with +40%, the held-out one with +16%. In Figure 5 the held-out estimate stops rising after about 750 steps, while the train estimate keeps climbing for the rest of the run. Everything gained after that point is visible only to the model being optimised.

No logged evidence supports the gain. Relevance on liked tracks falls as much as relevance on skipped ones. The policy did not learn to avoid what users skip; it left the region where the log says anything.

A good held-out score did not protect the reward model. It is accurate and calibrated on the events users generated. RL asks it about user-track pairs that never occurred, and there its item-level beliefs take over.

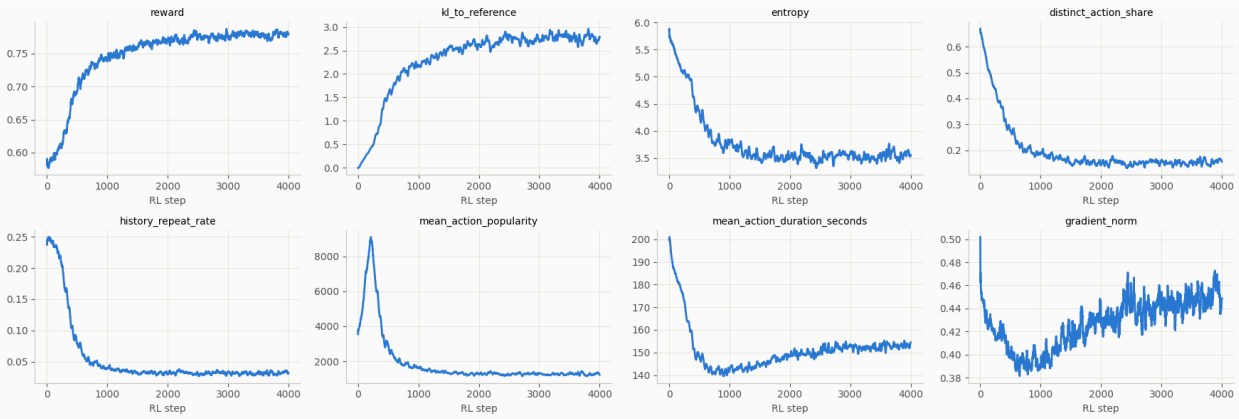


Figure 4: What the policy samples during the main RL run of run A (20-step moving average). The KL to the base recommender settles near 2.8 nats per position; the share of distinct tracks in a batch falls from 0.67 to 0.15. Popularity first rises, then falls far below its starting level.

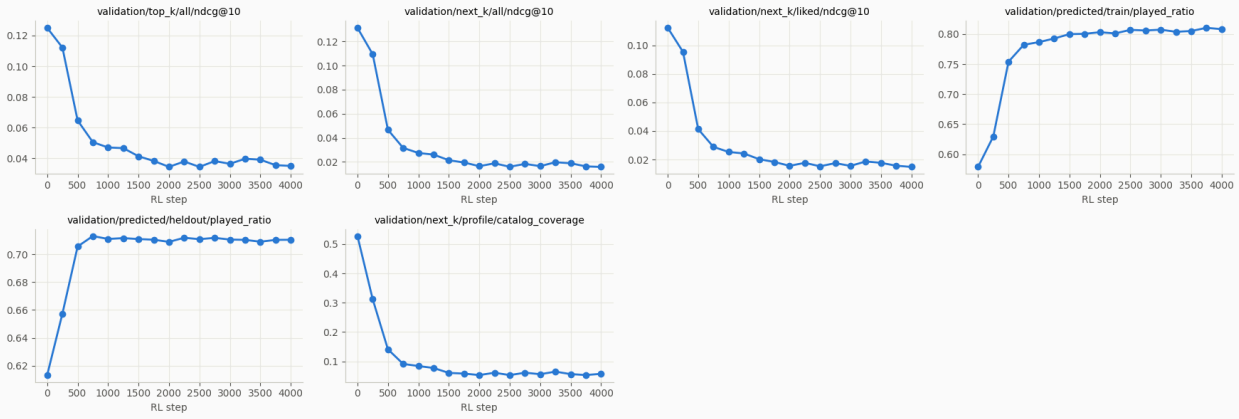


Figure 5: Greedy lists for validation users during the same run. Relevance is lost in the first 500 steps. The reward predicted by the held-out model plateaus early; the reward predicted by the train model keeps rising.

2.4 The anchor decides, and the transition is sharp

Table 6 repeats the fine-tuning at seven KL weights. Below 0.1 the policy degenerates completely: with no anchor all users receive the same ten tracks (coverage 0.0002 of 50,000 tracks, overlap 1.0). At 1 and above it barely moves. The interesting behaviour lies between 0.1 and 0.3, and this grid stepped over it.

Table 6: KL-weight sweep of run A: generated lists on test users after 2,000 RL steps. "Predicted" is the mean predicted played ratio of the list.

KL weight	Base	0	0.01	0.03	0.1	0.3	1	3
NDCG@10, all	0.1175	0.0007	0.0007	0.0006	0.0118	0.1155	0.1183	0.1176
NDCG@10, liked	0.1010	0.0007	0.0007	0.0005	0.0114	0.0998	0.1021	0.1014
Recall@10, skipped	0.0737	0.0002	0.0002	0.0001	0.0059	0.0691	0.0734	0.0734
Predicted, train model	0.580	0.830	0.830	0.836	0.806	0.611	0.591	0.585
Predicted, held-out model	0.613	0.728	0.728	0.701	0.712	0.633	0.619	0.615
Catalogue coverage	0.523	0.0002	0.0002	0.0004	0.046	0.472	0.514	0.520
Overlap of two users	0.001	1.000	1.000	0.895	0.596	0.005	0.002	0.002
Mean duration (s)	203	119	119	123	125	189	199	202
Share from history	0.505	0.005	0.005	0.003	0.034	0.479	0.505	0.507

At a weight of 0.3 the outcome is the one RL is meant to produce. NDCG@10 falls by 1.7% and NDCG on liked tracks by 1.2%, while recall of skipped tracks falls by 6.2%: the policy gives up more of what users skip than of what they play. The predicted played ratio rises by 5.3% according to the train reward model and by 3.3% according to the held-out one, so most of the gain is confirmed. Coverage stays at 0.47. Figure 6 shows the same sweep as a trade-off curve.

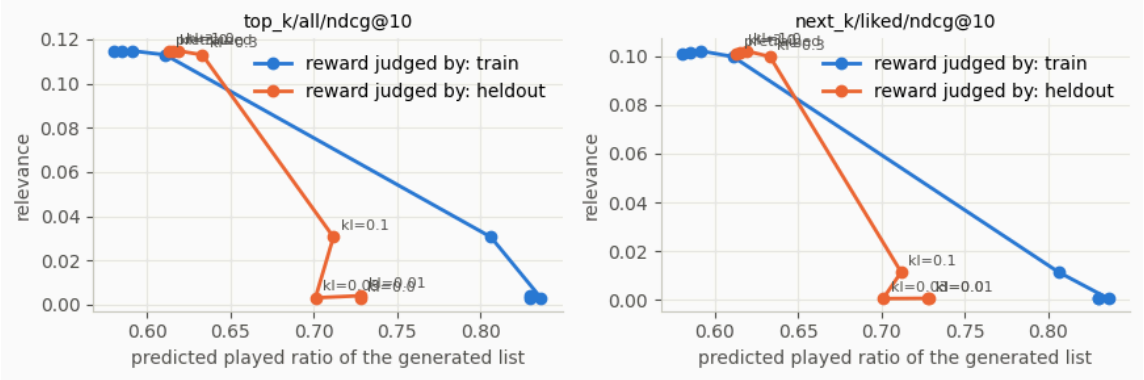


Figure 6: Relevance against predicted reward across the KL-weight sweep of run A. The horizontal distance between the two curves is the part of the reward that only the train reward model reports.

2.5 Each reward is gamed through the attribute it correlates with

Three other reward definitions were tried (Table 7). All of these runs used the KL weight of 0.1, which the sweep later showed to be on the collapsing side, so the table shows what each reward is gamed towards rather than a fair comparison of rewards. It has to be repeated at a weight that holds.

Table 7: Reward variants in run A, 2,000 RL steps each at KL weight 0.1. *Played time* multiplies the ratio by the track's relative duration; *session* is the predicted probability that the user keeps listening.

Reward	Base	Played ratio	Played time	Ratio + session	Session only
NDCG@10, all	0.1175	0.0118	0.0003	0.0105	0.0056
Predicted played ratio (held-out)	0.613	0.712	0.545	0.710	0.634
Predicted seconds played (held-out)	123	89	801	89	110
Predicted continuation (held-out)	0.937	0.941	0.869	0.941	0.941
Catalogue coverage	0.523	0.046	0.0004	0.040	0.025
Overlap of two users	0.001	0.596	0.960	0.555	0.111
Mean duration (s)	203	125	1,513	125	180

Rewarding the played ratio moves the policy to short tracks with high completion. Rewarding played time, which was meant to remove that incentive, moves it to 25-minute tracks instead and raises the predicted seconds played more than sixfold. Rewarding session continuation changes little in the predicted continuation itself (0.937 to 0.941), because 95% of events continue the session anyway; the signal has almost no room to move at a 30-minute gap. In run B, where the same variants were trained for only 500 smaller steps, *ratio + session* and *session only* lost 17% and 24% of NDCG@10 instead of nearly all of it, which says the collapse had not finished rather than that those rewards are safer.

2.6 Generating several steps ahead

Figure 7 follows the generated list position by position. For the base recommender the probability that a generated track is in the user's real continuation falls from 0.165 at the first position to 0.06 at the tenth, and the reward model's predicted played ratio falls with it, from 0.67 to 0.55: as the list moves away from the real history, the reward model expects less. For the fine-tuned policy the predicted ratio rises along the list, from 0.75 to above 0.82, and the two reward models disagree more from the fourth position on.

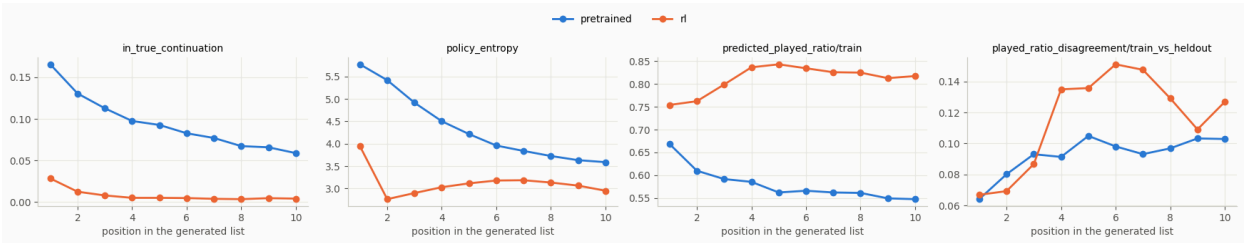


Figure 7: Generated lists by position, base recommender against the main RL run of run A (test users).

Part of that rise is produced by the simulation. A generated track has no observed feedback, yet the reward model needs one to score the next track. Figure 8 scores the same lists of the fine-tuned policy under three assumptions. At the tenth position the predicted played ratio is 0.82 if the model is fed its own expectation, 0.91 if it is fed a sampled skip-or-play outcome, and 0.96 if every generated track is assumed to have been played; the first position, which depends on real history only, is 0.755 in all three. The reward model has learned that a user who has just played tracks through will play the next one, and a policy that picks tracks with high predicted completion creates that context for itself. Multi-step predicted reward therefore measures the simulation as much as the recommendations.

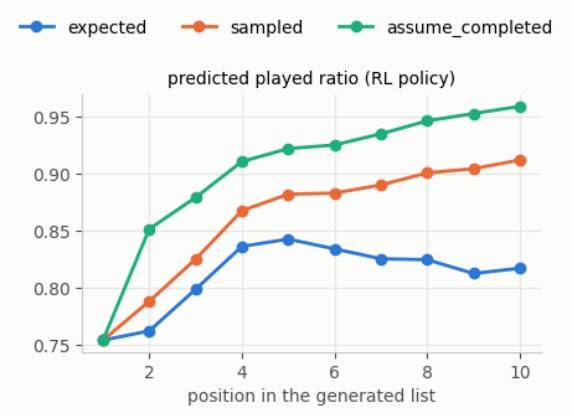


Figure 8: Predicted played ratio of the fine-tuned policy's lists under three assumptions about the feedback on already generated tracks (run A, train reward model).

2.7 Run B shows the same collapse at an earlier stage

Run B looked better at first sight: its main RL run lost 68% of NDCG@10 instead of 89%, and it finished in about half the time on a much weaker GPU. Neither is a contradiction. It ran in less time because it did far less work: 6 epochs instead of 15 on about half as many training windows over a smaller softmax, and 7 sweep runs of 500 steps instead of 11 of 2,000. Its policy was stopped while still collapsing. In Figure 9 relevance is falling at the last evaluation, and the KL to the base recommender was at 1.7 nats and rising when training ended, against the plateau of 2.8 in run A. Its main run saw about eleven times fewer sampled lists.

The sweep of run B agrees with run A where they overlap. After only 500 steps the runs at KL weights 0 and 0.03 had already collapsed (NDCG@10 0.0017 and 0.0043, overlap of two users 1.00 and 0.98), and the run at 0.3 held: NDCG@10 0.1237 against 0.1226 for its base recommender, recall of skipped tracks −2.4%, predicted played ratio +2.7% and +2.4% by the two reward models. Metrics of the two runs are not comparable in level, because the catalogues differ.

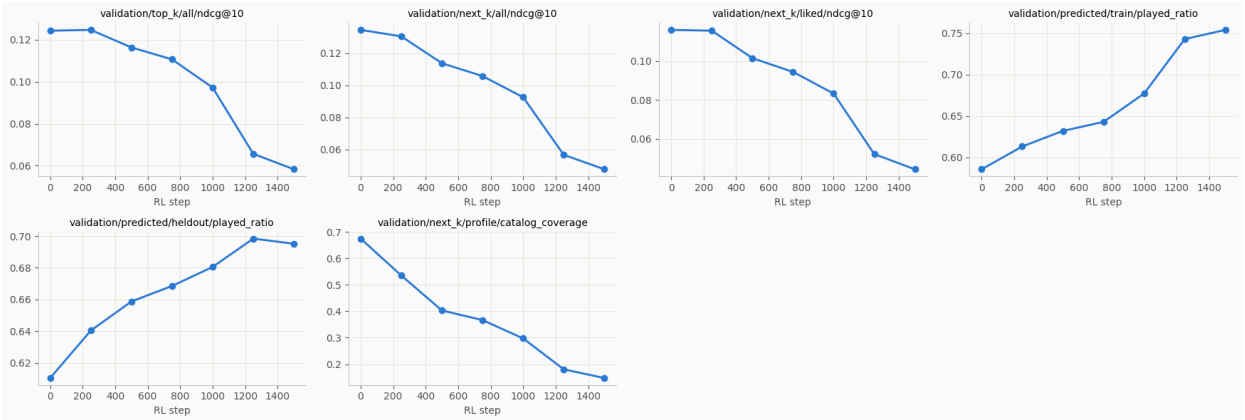


Figure 9: Run B, main RL run: greedy lists for validation users. The run ends at step 1,500 with relevance still falling.

3 KION with PPO (run C)

3.1 The base recommender against the baselines

Table 8 places the recommender among the baselines, all producing lists of ten titles the user has not watched. Three things stand out.

Popularity is a strong floor on KION. The ten most popular unseen titles reach NDCG@10 0.090, and the GPT recommender is 35% above that, where on Yambda it is six times above. The same is true of the dataset's own competition: the five best entries scored MAP@10 0.114 to 0.122 against 0.091 for popularity [4], a ratio of 1.25 to 1.34. Here the ratio in MAP@10 is 1.37 for the GPT

recommender and 1.46 for BERT4Rec. The protocols differ, so only the ratio is comparable, and it says the recommender is in the expected range.

The GPT recommender is a mid-table model here. It equals GRU4Rec, is 13% above SASRec and 5% below BERT4Rec. SASRec differs from the GPT recommender mainly by its loss, one sampled negative per position instead of a full softmax, so the gap between them is the price of that loss on this dataset. The gap to BERT4Rec, 0.007, is several times the bootstrap half-width found for the RL comparison below (0.001), but no interval was computed between baselines and every model was trained once.

Lists are concentrated before any RL. Every neural model recommends its most frequent title to about half of the users, and two users share 14% to 21% of their lists. On Yambda the same figures for the base recommender are 2% and 0.1%. Collapse measures on KION therefore start from a high level.

Generating the list item by item is slightly worse than ranking on KION (NDCG@10 0.117 against 0.122) and covers less of the catalogue. This is what Petrov and Macdonald [1] report and the opposite of Yambda. A likely reason is that Yambda users replay tracks, so a generated track is a plausible part of the history it joins, while a KION title is watched once.

Table 8: Baselines and the GPT recommender on 59,997 KION test users (run C). *Liked* titles are future titles watched at least half; *abandoned* ones are the rest.

	Most popular	**Markov chain**	**GRU4Rec**	**SASRec**	**BERT4Rec**	**GPT, ranking**	**GPT, generation**
NDCG@1	0.0597	0.0669	0.0877	0.0726	0.0945	0.0885	0.0885
NDCG@10	0.0903	0.0959	0.1215	0.1082	0.1290	0.1220	0.1173
Recall@10	0.1303	0.1394	0.1721	0.1580	0.1811	0.1726	0.1620
MAP@10	0.0607	0.0637	0.0824	0.0720	0.0885	0.0831	0.0819
Hit rate@10	0.2358	0.2571	0.3154	0.2903	0.3281	0.3144	0.2871
NDCG@10, liked titles	0.1003	0.1043	0.1294	0.1104	0.1320	0.1293	0.1214
Recall@10, abandoned titles	0.1078	0.1155	0.1436	0.1346	0.1551	0.1442	0.1382
Catalogue coverage	0.004	0.466	0.544	0.335	0.427	0.574	0.495
Users receiving the top title	0.918	0.530	0.502	0.504	0.496	0.484	0.566
Overlap of two users' lists	0.714	0.220	0.149	0.161	0.161	0.140	0.209
Predicted watched share (held-out)	0.473	0.473	0.454	0.452	0.438	0.455	0.444

3.2 The reward model

The KION reward models are weaker than the Yambda ones and still well ahead of the simple predictors (Table 9). The two copies agree: the correlation of their predictions on test events is 0.94. Calibration is good in the middle of the range and compressed at the ends: events predicted at about 0.06 are watched to 0.12, events predicted at about 0.93 to 0.87.

Two properties matter for RL. First, the title alone predicts almost as well as the user's recent behaviour (AUC 0.66 against 0.67), where on Yambda the user dominated (0.80 against 0.61). On KION the reward is to a large extent a property of the title, and a policy can raise it by choosing titles. Second, the models overpredict the watched share of the least popular fifth of the catalogue by 0.13 to 0.14, against 0.02 for popular titles. That is the weakness the Yambda policy exploited.

Table 9: Prediction of the watched share on the 184,209 test-period events of KION, each predicted from the events before it (run C). AUC is for "watched at least half".

Predictor of the watched share	MAE	RMSE	Cross-entropy	AUC
Train reward model	0.290	0.347	0.608	0.756
Held-out reward model	0.291	0.349	0.610	0.753
Mean of the user's preceding events	0.331	0.384	0.670	0.673
Mean watched share of the title	0.334	0.379	0.657	0.656
Global mean	0.370	0.399	0.689	0.500

3.3 PPO with an anchor that holds

The main run used a KL weight of 0.3. Table 10 compares the fine-tuned policy with the base recommender.

Table 10: Main RL run of run C (PPO, KL weight 0.3, 2,000 iterations) against the base recommender. Intervals are in absolute units, from a paired bootstrap over users.

Generated lists, test users	Base	After RL	Change	95% interval of the change
NDCG@10, all future titles	0.1173	0.1039	−11.4%	−0.0144 to −0.0124
NDCG@10, liked titles	0.1214	0.1228	+1.2%	0.0000 to +0.0028
Recall@10, abandoned titles	0.1382	0.1112	−19.6%	−0.0289 to −0.0250
Predicted watched share, train reward model	0.433	0.536	+23.9%	+0.1029 to +0.1041
Predicted watched share, held-out reward model	0.444	0.545	+22.8%	+0.1004 to +0.1016
Predicted seconds watched, held-out reward model	6,361	5,632	−11.5%	
Predicted continuation, held-out reward model	0.804	0.806	+0.3%	
Catalogue coverage	0.495	0.434		
Share of users receiving the top title	0.566	0.554		
Expected overlap of two users' lists	0.209	0.275		
Mean popularity of recommended titles (viewers)	37,483	40,467		
Mean duration of recommended titles (hours)	5.7	3.2		
Mean watched share of recommended titles in the log	0.457	0.560		

This is the outcome the method is meant to produce. Five observations support or qualify it.

The gain is confirmed. The held-out reward model reports +22.8% where the train one reports +23.9%, and in Figure 10 the two rise together and flatten after about 500 iterations. The sweep run at the same weight, stopped at 1,000 iterations, ends at the same point. Nothing here is visible only to the model being optimised.

The logged evidence agrees. Relevance on titles the user went on to watch at least half of is intact, and its interval starts at zero. Relevance on titles the user abandoned falls by a fifth. The policy gives up what users drop and keeps what they finish.

What it moved to. The titles that gained the most exposure are films of 85 to 130 minutes with a mean watched share of 0.54 to 0.76; the largest gainer went from the lists of 8,388 users to 26,169 of the 59,997. The titles that lost the most are series: one with 17 hours of content and a mean watched share of 0.08 fell from 33,948 lists to 8,045, another from 16,278 to 558. The mean duration of a recommended title fell from 5.7 hours to 3.2. Two niche films with 953 and 175 viewers in the training period also rose, to 20,403 and 15,901 lists; that is the Yambda pattern in a small dose.

The reward says less than it seems. The watched share of a series is taken of the whole series, so a series is rarely "liked" by the half-watched rule and scores low for everyone. Both the reward gain and the split between liked and abandoned titles therefore largely restate one fact, that the policy now prefers films. The predicted time watched falls by 11.5%. Whether a finished film is worth more than a started series is a product question the watched share answers by construction.

How PPO behaved. The policy stayed close to the base recommender: the KL settled near 0.10 nats per position, against 2.8 in run A. The share of actions outside the clip range was zero throughout, so the clip never acted; the small learning rate and the anchor kept each update inside it. The value

model explained about half of the variance of the returns. As run, this is advantage actor-critic with generalised advantage estimation and four passes over each batch.

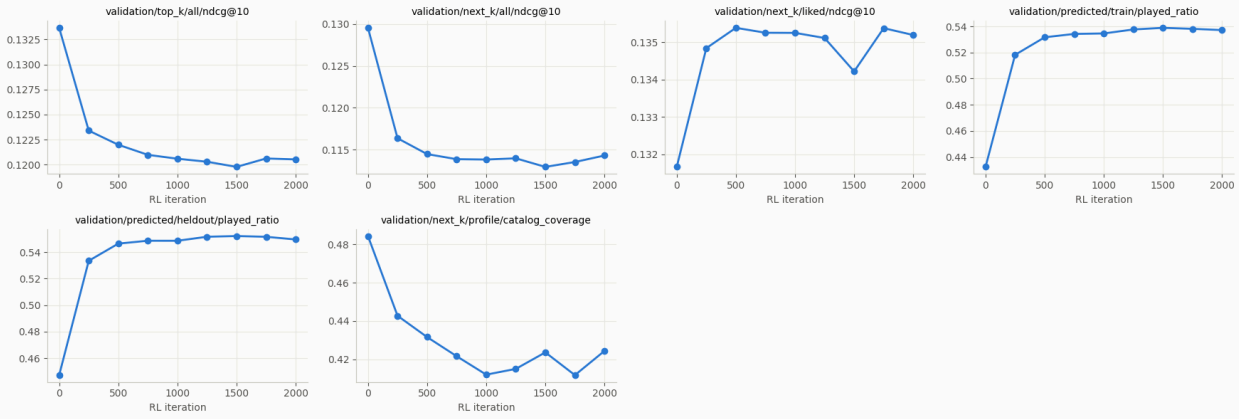


Figure 10: Run C, main RL run: greedy lists for validation users. Relevance on all titles falls and relevance on liked titles rises within the first 500 iterations; the reward predicted by both reward models flattens at the same time.

3.4 The trade-off is smooth

Table 11 repeats the fine-tuning at six KL weights, and Figure 11 draws it as a curve.

Table 11: KL-weight sweep of run C: generated lists on test users after 1,000 PPO iterations. "Predicted" is the mean predicted watched share of the list.

KL weight	**Base**	**0**	**0.1**	**0.2**	**0.3**	**0.5**	**1**
NDCG@10, all	0.1173	0.0007	0.0736	0.0987	0.1042	0.1114	0.1153
NDCG@10, liked	0.1214	0.0012	0.0944	0.1206	0.1240	0.1261	0.1250
Recall@10, abandoned	0.1382	0.0004	0.0654	0.1006	0.1099	0.1280	0.1350
Predicted, train model	0.433	0.871	0.653	0.559	0.534	0.499	0.467
Predicted, held-out model	0.444	0.748	0.652	0.570	0.544	0.509	0.477
Predicted seconds (held-out)	6,361	4,511	4,101	5,675	5,775	6,006	6,098
Catalogue coverage	0.495	0.006	0.248	0.386	0.423	0.451	0.474
Overlap of two users	0.209	0.991	0.377	0.317	0.280	0.250	0.226
Mean duration (hours)	5.7	1.7	1.9	3.0	3.3	4.1	4.8

Every loosening of the anchor buys predicted reward with relevance, and no step is a cliff. From weight 1 down to 0.2 the reward gain grows from 8% to 28% while relevance on liked titles stays within 4% of the base, mostly above it; what is given up is relevance on abandoned titles, down by 2% to 27%. Below 0.2 the liked titles go as well: at 0.1 the reward is up 47%, NDCG@10 on liked titles is down 22% and overall NDCG@10 is down 37%. With no anchor the policy collapses as it did on Yambda: 46 titles are ever recommended and two users share 99% of their lists.

The two reward models agree at every weight except zero, where the train model reports 0.87 and the held-out one 0.75. Exploitation of the reward model appears only at the extreme.

At KL weight 0.1 the Yambda policy lost 89% of NDCG@10 and the KION policy 37%. This cannot be credited to PPO: the dataset, the learning rate and the number of updates changed with it, and PPO with no anchor collapses just the same.

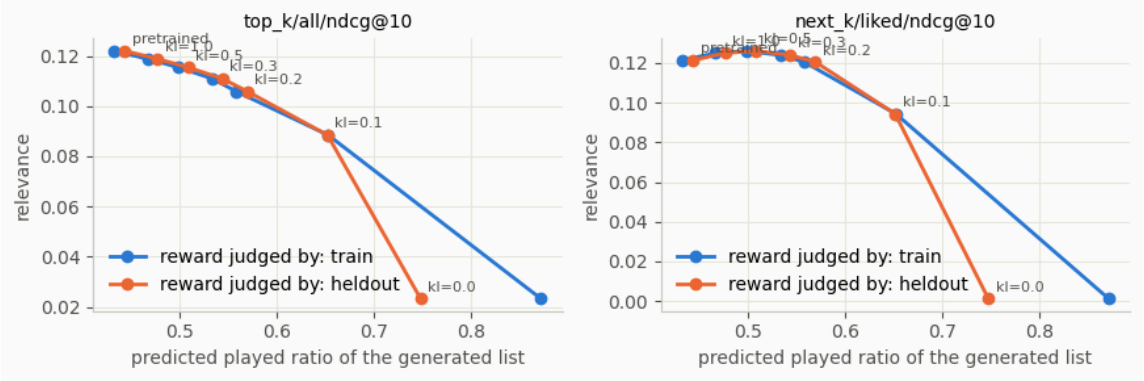


Figure 11: Relevance against predicted reward across the KL-weight sweep of run C. The two curves coincide except at weight zero.

3.5 Does it take RL?

Table 12 places the two cheaper methods next to the RL runs, and Figure 12 draws all of them on the plane of relevance and reward predicted by the held-out model.

Table 12: RL against liked-only fine-tuning and re-ranking of 100 candidates on KION test users (run C). RL and liked-only lists are generated; re-ranked lists start from the ranking of the base recommender.

Lists of ten	NDCG@10, all	NDCG@10, liked	Recall@10, abandoned	Predicted (held-out)	Coverage	Duration (hours)
Base recommender, generation	0.1173	0.1214	0.1382	0.444	0.495	5.7
Base recommender, ranking	0.1220	0.1293	0.1442	0.455	0.574	6.2
RL, KL weight 1	0.1153	0.1250	0.1350	0.477	0.474	4.8
RL, KL weight 0.5	0.1114	0.1261	0.1280	0.509	0.451	4.1
RL, KL weight 0.3 (main run)	0.1039	0.1228	0.1112	0.545	0.434	3.2
RL, KL weight 0.2	0.0987	0.1206	0.1006	0.570	0.386	3.0
RL, KL weight 0.1	0.0736	0.0944	0.0654	0.652	0.248	1.9
Liked-only fine-tuning	0.1036	0.1258	0.1017	0.528	0.473	3.0
Re-ranking, $w = 1$	0.1198	0.1335	0.1377	0.485	0.567	4.6
Re-ranking, $w = 0.3$	0.1110	0.1335	0.1194	0.534	0.530	3.2
Re-ranking, $w = 0.1$	0.0927	0.1211	0.0878	0.592	0.435	2.4
Re-ranking, $w = 0.03$	0.0722	0.0977	0.0667	0.620	0.346	2.1
Re-ranking, $w = 0$	0.0545	0.0745	0.0528	0.626	0.329	2.0

Re-ranking is at least as good as RL up to a reward of about 0.59. At $w = 0.3$ it reaches a predicted watched share of 0.534 with NDCG@10 0.111 overall and 0.134 on liked titles. The RL run nearest in reward (the main run, 0.545) has 0.104 and 0.123; the RL run nearest in relevance (weight 0.5, NDCG@10 0.111) has a reward of 0.509. At $w = 1$ re-ranking improves on the base recommender's generated lists on every axis at once. Re-ranked lists cover more of the catalogue than RL lists at the same reward, but give their most frequent title to more users (0.58 at $w = 0.3$ and 0.67 at $w = 0.1$, against 0.55 for the main RL run). Part of the advantage is the starting point: re-ranking starts from the ranked list, which on KION is 0.005 NDCG@10 better than the generated one. The RL policy's own ranked lists score 0.110 overall and 0.130 on liked titles, level with re-ranking at $w = 0.3$, but their predicted reward was not measured.

Liked-only fine-tuning lands on the main RL run. One epoch of supervised training on items watched at least half gives the same overall NDCG@10 (0.104), slightly higher relevance on liked titles, lower recall of abandoned ones, and a predicted reward of 0.528 against 0.545. It needs no reward model. This stage and all the re-ranking took 3.4 minutes together; the main RL run took 14.3.

RL reaches further. Re-ranking 100 candidates cannot exceed a predicted watched share of 0.626, and near that limit its relevance falls steeply. RL at KL weight 0.1 reaches 0.652 with NDCG@10

0.074, above the 0.055 to 0.072 of re-ranking near its limit. This is the only region where RL is the better tool, and it is the region where relevance on liked titles is already down 22%.

The likely reason is the reward. The predicted watched share is a sum of per-item terms, and on KION it is mostly a property of the title. Choosing items with a high predicted reward one at a time is then close to optimal, and that is what a re-ranker does. RL should have an edge when the reward depends on the list as a whole, which this one does only through the simulated feedback on earlier items.

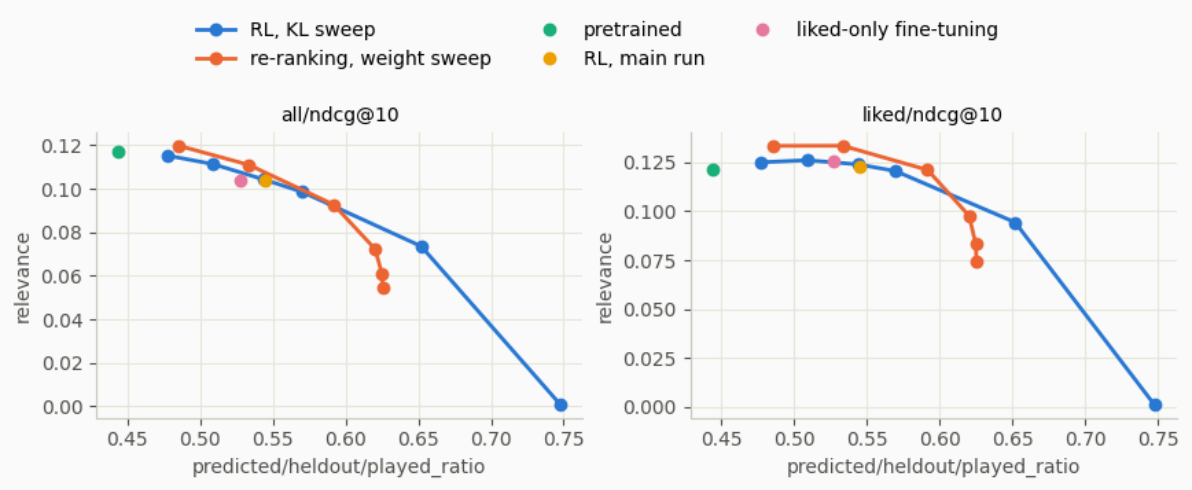


Figure 12: Relevance against the reward predicted by the held-out model on KION (run C). The rightmost point of the RL curve is the run with no anchor.

3.6 Reward variants at an anchor that holds

On Yambda the reward variants were compared at a weight that collapses. Here all of them ran at 0.3, and none collapsed (Table 13).

Table 13: Reward variants in run C, 1,000 PPO iterations each at KL weight 0.3. *Watch time* multiplies the share by the title's relative duration; *session* is the predicted probability that the user watches another title within 7 days.

Reward	Base	Watched share	Watch time	Share + session	Session only
NDCG@10, all	0.1173	0.1042	0.0885	0.1098	0.1093
NDCG@10, liked	0.1214	0.1240	0.0901	0.1217	0.1131
Recall@10, abandoned	0.1382	0.1099	0.1042	0.1281	0.1324
Predicted watched share (held-out)	0.444	0.544	0.307	0.487	0.440
Predicted hours watched (held-out)	1.77	1.60	10.51	1.62	1.61
Predicted continuation (held-out)	0.804	0.805	0.763	0.812	0.821
Catalogue coverage	0.495	0.423	0.274	0.487	0.505
Overlap of two users	0.209	0.280	0.304	0.189	0.144
Mean duration (hours)	5.7	3.3	63.6	4.3	5.1

Rewarding watch time, meant to remove the preference for short titles, overshoots as it did on Yambda. The policy moves to long series, the mean duration of a recommended title rises from 5.7 hours to 64, the predicted time watched rises sixfold, and the predicted watched share, the continuation and a quarter of the relevance are lost. Adding the session term to the watched share halves the shift: the share rises by 10% instead of 23%, relevance on liked titles is unchanged and overall NDCG@10 falls by 6%. Rewarding the session alone raises the predicted probability of another viewing within a week from 0.804 to 0.821 and leaves the watched share where it was. It costs 7% of NDCG@10, on liked titles as much as on all, and it is the only variant that makes lists less alike: the overlap between two users falls from 0.21 to 0.14 and coverage rises. A weekly return signal has more room than the 30-minute one on Yambda, but 1.7 points for 7% of relevance is a weak lever.

3.7 Generating several steps ahead

Figure 13 follows the generated list position by position. The probability that a generated title is in the user's real continuation falls from 0.088 at the first position to 0.016 at the tenth for the base recommender, and from 0.081 to 0.018 after RL. Generation falls behind ranking as the list grows, and more so after RL (Table 14).

Table 14: Ranking against item-by-item generation on KION test users (run C). The first item is the same by construction.

NDCG at	Base, ranking	Base, generation	RL, ranking	RL, generation
1	0.0885	0.0885	0.0812	0.0812
5	0.1055	0.1044	0.0943	0.0911
10	0.1220	0.1173	0.1103	0.1039

The predicted watched share falls along the list for both policies, from 0.53 to 0.38 for the base recommender and from 0.62 to 0.43 after RL. The advantage of the fine-tuned policy shrinks from ten points at the first position to five at the tenth. The two reward models disagree by 0.052 to 0.057 at every position for both policies; RL does not push the list into a region where they part.

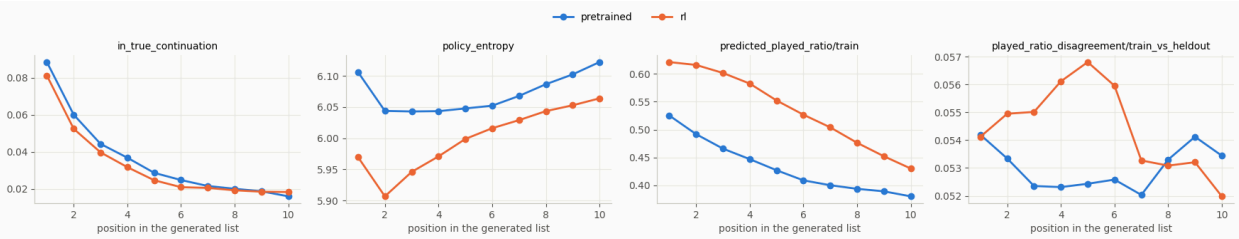


Figure 13: Generated lists by position, base recommender against the main RL run of run C (test users).

As on Yambda, much of the profile along the list is produced by the simulation (Figure 14). At the first position the predicted watched share of the fine-tuned policy is 0.62 whatever is assumed. At the tenth it is 0.43 if the reward model is fed its own expectation for the titles generated before, 0.53 if it is fed a sampled outcome, and 0.68 if every generated title is assumed to have been watched to the end: a spread of 25 points. The expectation is the most pessimistic assumption here, and the reason is likely the input it creates. Real watched shares are near 0 or near 1; a history of titles each watched to about 0.5 is one the reward model has hardly seen. RL was trained with that assumption, so the level of its multi-step reward, and the decline in Figure 13, are partly artefacts of it.

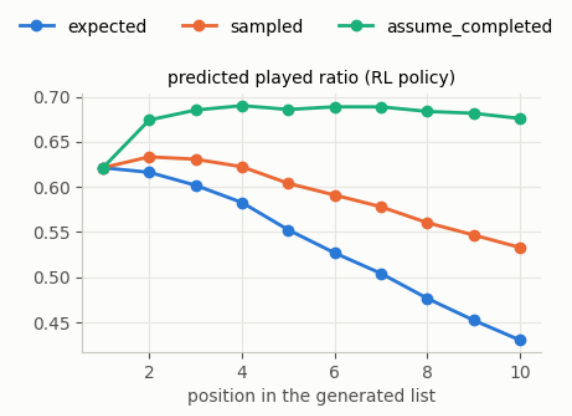


Figure 14: Predicted watched share of the fine-tuned policy's lists under three assumptions about the feedback on already generated titles (run C, train reward model).

4 Comparison with the two papers

The papers report RL that preserves relevance. Here the first main run did not, and the third did at the price of 11% of overall NDCG@10. The difference from the papers is not in the optimiser but in where the reward comes from and what data the policy is updated on (Table 15).

Table 15: How the three setups differ. Figures for [1] are NDCG@10 on MovieLens-1M and Steam-2M across their reported settings; figures for [2] are their Tables 1 and 3. Yambda figures are from REINFORCE, KION figures from PPO.

	Petrov and Macdonald [1]	**Matveev et al. [2]**	**This work**
Reward	NDCG against the true next item, plus a diversity or popularity term	Logged like and played ratio of logged recommendations	Played ratio predicted by a learned model
Policy is updated on	Its own sampled lists	Logged sessions, importance weights capped at 0.5	Its own sampled lists
What keeps it relevant	The NDCG term inside the reward	It can only reweight logged actions	A KL penalty to the base recommender
Role of a learned reward model	None	Evaluation only	The training signal
Relevance after RL	NDCG@10 +3% to −38%, by the weight of the second term	Recall@100 −2%; −6% on the lowest-return actions	NDCG@10 at KL weight 0.3: −2% on Yambda, −11% on KION. At 0.1: −89% and −37%
Reward after RL	The target metric improves	Model-based return +2% to +4%	Predicted played ratio at 0.3: +3% to +5% on Yambda, +23% on KION

Three points follow. First, in the regime that resembles theirs, a strong anchor, the Yambda result agrees with Matveev et al. almost number for number: a relevance loss near 2%, a loss roughly three times larger on the interactions users did not want, and a reward gain of a few percent. KION shows the same shape at a larger size: −11% overall, −20% on abandoned titles, no loss on liked ones, +23% reward. Second, both papers also lose relevance when the constraint is loosened. Petrov and Macdonald lose 15% of NDCG@10 on MovieLens-1M and 38% on Steam-2M at their strongest popularity penalty; Matveev et al. lose 9% of recall when the cap on importance weights is raised from 0.5 to 10. What they cannot do is collapse onto ten tracks, because one keeps ground-truth relevance in the reward and the other never leaves the logged actions. The setup studied here, in which a learned model is the only source of reward, removes both protections. Matveev et al. make the same caution about their own model-based numbers: results selected and evaluated with the same feedback model are not independent evidence. Third, using the optimiser of Petrov and Macdonald did not change the picture. PPO with no anchor collapsed as REINFORCE did, and with an anchor its clip never acted.

5 What the five questions look like so far

1. **Does behaviour change, and is the reward maximised?** Yes to both, on both datasets. The predicted reward rises with every loosening of the anchor, and the policy follows the item attribute the reward correlates with. On Yambda it moves from the user's own repertoire to short, obscure tracks with high logged completion, and the held-out reward model confirms less than half of the gain. On KION it moves from series to films, and the held-out model confirms the gain at every anchor weight above zero.
2. **Is relevance preserved?** With an anchor that holds. On Yambda at KL weight 0.3 it is nearly intact, and at 0.1 it is gone and most users receive the same tracks. On KION at 0.3 overall NDCG@10 falls by 11%, all of it on titles users abandon; relevance on titles they finish is kept down to 0.2. Lists on KION are concentrated to begin with, and RL at 0.3 adds moderately to that: the overlap between two users rises from 0.21 to 0.27 and coverage falls from 0.50 to 0.43. With no anchor everyone receives the same items on both datasets.
3. **Long-term goals.** Session continuation at a 30-minute gap on Yambda is too saturated (95%) to be a reward. Watching another title within a week on KION (80%) has more room: optimising it raised its predicted probability by 1.7 points for 7% of relevance and made lists less alike. That is a weak lever, and a predicted one. Offline one can check whether such a signal is predictable and what optimising it does to the lists; whether users actually return needs an online test.

4. **Several steps ahead.** Lists drift from the real continuation with every position, and on KION generating is already worse than ranking. The predicted reward of later positions depends on the assumed feedback by up to 14 points on Yambda and 25 on KION. A multi-step reward estimate from this simulator should not be reported without that caveat.
5. **The trade-off.** It exists on both datasets. On Yambda with REINFORCE it is abrupt in the anchor weight; on KION with PPO it is smooth, and the reward definition moves it: a session term halves the shift. On KION the same curve is reached or bettered by re-ranking with the reward model up to a reward gain of about 30%. There the trade-off belongs to the reward model and the recommender more than to RL.

6 Limitations of this iteration

- One seed per configuration. Intervals exist only for the comparison with the base recommender on KION, and they cover the user sample, not the variation between training runs. Baselines have no intervals.
- Dataset and optimiser are confounded. Yambda ran only with REINFORCE and without baselines or alternatives; KION ran only with PPO.
- The baselines are not tuned, and SASRec uses its sampled loss. Replaying the user's own history, the natural baseline for music, was added after the code for run C was packaged and has not run.
- The clip of PPO never acted, so run C does not test the part of PPO that distinguishes it.
- Re-ranking and RL are not on an equal footing: re-ranking starts from ranked lists and 100 candidates, RL is judged on generated lists, and the predicted reward of the RL policy's ranked lists was not measured.
- On KION the watched share is tied to the length of the title, and with it the reward and the split into liked and abandoned titles.
- On Yambda the reward variants were compared at a KL weight that collapses.
- The reward models were validated on logged events. On Yambda that was not done separately for recommended events, which is what RL asks about; on KION nothing marks them.
- Advantages were normalised, which neither paper does; the KL weights are therefore not on their scale.
- Everything is offline. No number here says how real users would respond to the fine-tuned policy.

7 Next iteration

- **Yambda with the pipeline of run C**: PPO at KL weight 0.3, the baselines including replay of the user's history, the alternatives to RL and the intervals. It is configured and not yet run. It separates the dataset from the optimiser, and it tests whether re-ranking still matches RL where the reward is less a property of the item.
- **An equal footing for re-ranking and RL**: the predicted reward of the RL policies' ranked lists, and re-ranking with 300 and 1,000 candidates.
- **A reward that does not encode length**: the watched share relative to what is usual for the title, or for its type. Rewarding watch time overcorrects on both datasets.
- **A reward that depends on the whole list**, which is where RL should beat a re-ranker.
- **Three seeds** for the base recommender, the baselines and the main RL run.
- **PPO in a regime where its clip acts**: a larger learning rate with a weaker anchor, to see whether the clip can carry part of what the anchor does now.

Three changes to the reward itself remain candidates: penalising the disagreement between two reward models, adding a relevance term as in Petrov and Macdonald, and withholding simulated feedback on generated items from the reward model. The 25-point spread on KION makes the last one the most pressing.

References

1. A. V. Petrov and C. Macdonald. Aligning GPTRec with Beyond-Accuracy Goals with Reinforcement Learning. arXiv:2403.04875, 2024.
2. A. Matveev, S. Makeev, A. Krasilnikov, V. Baikalov, S. Liamaev and K. Khrylchenko. Session-Level Optimization for Large-Scale Retrieval using REINFORCE with Multi-Step Off-Policy Correction. arXiv:2607.02818, 2026.
3. A. Ploshkin et al. Yambda-5B: A Large-Scale Multi-modal Dataset for Ranking and Retrieval. arXiv:2505.22238, 2025.
4. A. Petrov, I. Safilo, D. Tikhonovich and D. Ignatov. MTS Kion Implicit Contextualised Sequential Dataset for Movie Recommendation. arXiv:2209.00325, 2022. Competition results: the README of `github.com/irsafilo/KION_DATASET`.

Appendix: where the time went

Table 16: Wall-clock minutes per stage. In run A the three RL stages are 81% of the total, at about one second per step of 1,024 lists. In run C a PPO iteration of 512 lists takes 0.43 seconds, on a catalogue six times smaller and windows of 48 instead of 128.

Stage	**Run A (min)**	**Run B (min)**	**Run C (min)**
Base recommender	99	109	4
Reward models	6	22	1
Baselines			5
RL main run	74	56	14
KL sweep	249	61	33
Alternatives to RL			3
Reward sweep	142	46	16
Everything else	1	6	3
Total	570	300	79